

AP(02) - Parte A e B**1s/UFSM****Período: 12-19/08/24****Acadêmico: André Gonçalves da Silva****Matrícula: 202210071****Curso/Turma: 123/11**

As questões da Avaliação Parcial 02 (AP(02)) tem por finalidade a aplicação direta dos aspectos teóricos e práticos dos métodos numéricos abordados na disciplina (Integração Numérica, Ajuste Funcional e Resolução de EDO(s) e recursos de programação em linguagem PYTHON 3 empregados nas respectivas implementações e cálculo numérico). A produção do arquivo-resposta seguirá os moldes da produção dos arquivos dos enunciados, nos formatos IPYNB e PDF e ambos deverão ser entregues via TAREFA aberta no MOODLE para a AP(02) de 19/08/2024.

Observa-se que a Parte A dessa avaliação substitui as mini-avaliações QUESTÃO (01), QUESTÃO (02) e QUESTÃO(03), substituição decorrente a excepcionalidade da alteração do calendário acadêmico de 2024.

OBSERVAÇÕES

- Salienta-se que observância da nomeação dos arquivos IPYNB e PDF (ver abaixo) e informações do cabeçalho (preencher devidamente) serão partes constituinte do escore (divisões sucessivas por 2):
 - 24_1s_MTM1013T11_AP02_AeB_NomeSobrenome.ipynb;
 - 24_1s_MTM1013T11_AP02_AeB_NomeSobrenome.pdf;
- Os algoritmos que devem ser empregados para as implementações em linguagem PYTHON 3 e *testes de mesas* são os algoritmos dados pelo material didático disponibilizado nos repositórios [MEGA](#) e [DRIVE](#) e não os sendo as respostas serão desconsideradas. Ressalta-se que os algoritmos do Material Didático são adequações dos algoritmos apresentados em Campos Filho (2018)[⁰], os quais podem ser utilizados para a implementação ou da Burden e Fayres (2016)[¹];

- A elaboração das questões buscará caracterizações personalizadas para cada aluno, fato que resultará em um diversificado conjunto de situações-testes para a exploração dos aspectos teóricos e práticos da disciplina em detrimento, eventualmente, da aplicabilidade do tema abordado na questão.
- Caso não seja mencionado, aplica-se o convencionado em aula:
 - Precisão de duas casas decimais ($\epsilon = 0,01$);
 - Erro Relativo: $E_R = \frac{|x_{k+1} - x_k|}{|x_{k+1}|}$;
- Para a elaboração de gráficos pode-se usar as bibliotecas [MATPLOLIB](#), [SYMPY](#) ou [SYMPY\(2\)](#). Ainda, a biblioteca NumPy (estende-se para SciPy, Pandas, ...) pode/deve ser empregada para a elaboração de gráficos e, para qualquer outra finalidade, quando no enunciado do exercício for especificado o uso ou quando não for substituinte do objetivo fim do exercício;
- Bibliografia: Há exemplares na BSCCNE.

[⁰]: Filho, Frederico Ferreira C. Algoritmos Numéricos - Uma Abordagem Moderna de Cálculo Numérico, 3ª edição. Disponível em: [Minha Biblioteca](#), Grupo GEN, 2018.

[¹]: Burden, Richard, L. et al. Análise Numérica - Tradução da 10ª edição norte-americana. Disponível em: [Minha Biblioteca](#), (3rd edição). Cengage Learning Brasil, 2016.

Obtenção de dados para elaboração das questões.

Ao considerar o meu número de matrícula na UFSM, 3332882, monta-se a tabela

d1	d2	d3	d4	d5	d6	d7	d8	d9	d10
0	0	0	3	3	3	2	8	8	2

e sobre essas informações, colocam-se as seguintes elaborações:

(I) Dedução dos parâmetros.

- $a = d5 = 3$ - caso $a = 0$, colocar $a = +1$;
- $b = d6 = 3$ - caso $b = 0$ ou $b = a$, colocar $b = +1$;

- $c = (d1 + d2 + d3 + d4) = (0 + 0 + 0 + 3) = 3$;
- $d = (d7 + d8 + d9 + d10) = (2 + 8 + 8 + 2) = 20 \xRightarrow{d \geq 10} d = (2 + 0) = 2$;
- $Dados = [a, b, c, d] = [3, 1, 3, 2]$;
- $r_0 = \min\{a, b, c, d\} = 1$;

(II) Sabe-se do estudo de fluidos que a quantidade de fluido por unidade de tempo $[(u. c.)^3 / (u. t.)]$ que atravessa esta secção uma secção circular de raio $r_0 = a$ é dado por:

$$Q = \frac{V}{t};$$

O volume pode ser descrito $V = Ad$, com A a área da secção transversal circular e d a profundidade desse quantidade que se quer medir ([ver ilustração](#))[²], logo

$$Q = \frac{V}{t} = \frac{Ad}{t} = A \frac{d}{t} = Av_{fluido},$$

com $v_{fluido} = d/t$ a velocidade do fluido;

Sobre essa análise qualitativa do processo de medição da quantidade de fluido que passa pela secção transversal circular, pode-se montar formulações mais precisas para a quantificação dessa quantidade Q , como segue:

- Da disciplina de Cálculo B (ou II) problemas associados à quantificação do fluxo de um fluido através de uma superfície S é dada pela integral de superfície:

$$\begin{aligned} \text{FLUXO} &= \iint_S \mathbf{v}(x, y, z) \cdot \mathbf{n} dS = \iint_{D_{xz}} (v_x, v_y, v_z) \cdot \frac{\overbrace{\mathbf{N}}^{\mathbf{n}}}{\|\mathbf{N}\|} \underbrace{\|\mathbf{N}\| dA}_{dS}, \\ &= \iint_{D_{xz}} (v_x, v_y, v_z) \cdot (n_x, n_y, n_z) dA = \iint_{D_{xz}} (v_x n_x + v_y n_y + v_z n_z) dA \end{aligned}$$

cujá resolução dará a quantidade de fluido que passa pela secção por unidade de tempo;

- Considere o campo de velocidades do fluido dado por $\mathbf{v}(x, y, z) = \left(0, b\left(1 - \frac{\sqrt{x^2+z^2}}{r_0}\right)^{1/7}, 0\right)$, com o centro do tubo de raio r_0 alinhado com o vetor normal $\mathbf{n} = (n_x, n_y, n_z) = (0, 1, 0) = \mathbf{j} = \frac{\mathbf{N}}{\|\mathbf{N}\|}$ a superfície de corte seccional descrita acima;
- Devido a simetria circular da seção transversal, emprega-se coordenadas polares para a descrição do campo de velocidades e no cálculo da quantidade de fluxo, ie,

$$\blacksquare \mathbf{v}(x, y, z) = \left(0, b\left(1 - \frac{\sqrt{x^2+z^2}}{r_0}\right)^{1/7}, 0\right) \equiv \left(0, b\left(1 - \frac{r}{r_0}\right)^{1/7}, 0\right) \Rightarrow v_y(r, \theta) = v(r) = b\left(1 - \frac{r}{r_0}\right)^{1/7};$$

- Conforme dados acima, segue que

$$\begin{aligned} Q &= \iint_{D_{xz}} v_y(x, z) dA, \quad D_{xz} := \{(x, z) \in \mathbb{R}^2 | 0 \leq x^2 + z^2 \leq r_0\} \\ &\equiv \iint_{D_{r\theta}} v_y(r, \theta) dA, \quad D_{r\theta} := \{(r, \theta) \in \mathbb{R}^2 | 0 < r < r_0 \wedge 0 \leq \theta < 2\pi\} \quad (x = r \cos \theta, z = r \sin \theta) . \\ &= \int_0^{r_0} \int_0^{2\pi} v_y(r, \theta) r dr d\theta = \int_0^{2\pi} d\theta \int_0^{r_0} r v_y(r, \theta) dr = 2\pi \int_0^{r_0} r v_y(r, \theta) dr = 2\pi \int_0^{r_0} r v(r) dr \end{aligned}$$

- Os itens acima, com a lista de de Dados aplicáveis:

$$\begin{aligned} \blacksquare \mathbf{v}(r, y, \theta) &= \left(0, b\left(1 - \frac{r}{r_0}\right)^{1/7}, 0\right) = \left(0, \left(1 - \frac{r}{1}\right)^{1/7}, 0\right) = \left(0, (1 - r)^{1/7}, 0\right); \\ \blacksquare D_{xz} &:= \{(x, z) \in \mathbb{R}^2 | 0 \leq x^2 + z^2 \leq r_0\} \equiv D_{xz} := \{(x, z) \in \mathbb{R}^2 | 0 \leq x^2 + z^2 \leq 1\}; \end{aligned}$$

(III) Gere as tabelas, com a seguinte caracterização para a função $v(r)$ e o integrando $rv(r)$:

- (A) Os sete primeiros pontos igualmente afastados para o intervalo $[0; 0, 8r_0]$;
- (B) Os cinco últimos pontos igualmente afastados para o intervalo $[0, 8r_0; r_0]$;

E a partir destas listas montadas, plotar estes pares ordenados e as funções geratrizes via biblioteca MATPLOTLIB;

Obs.: Os dados obtidos formarão uma tabela no formato:

r	0	...	$0,8r_0$	...	r_0
$v(r)$	b	...	$v(0,8r_0)$	...	$v(r_0)$
$rv(r)$	0	...	$(0,8r_0)v(0,8r_0)$	...	$r_0v(r_0)$

E os gráficos via WolframAlpha, para os dados acima:

- Gráfico de $v(r)$;
- Gráfico do integrando $rv(r)$;

(IV) A partir de (I), montar o PVI:
$$\begin{cases} ay' = b + \frac{cy}{dx}, x \in [1, 2] \\ y(1) = 2 \ (h = 0,05) \end{cases} \Leftrightarrow \begin{cases} 3y' = 1 + \frac{3y}{2x} \\ y(1) = 2 \end{cases}$$

[2]: KHAN ACADEMY. O que é taxa de fluxo de volume? Disponível em: <<https://pt.khanacademy.org/science/physics/fluids/fluid-dynamics/a/what-is-volume-flow-rate>>. Acesso em: 12/08/2024.

QUESTÕES - PARTE A

1) (1,0 ponto) Faça o que se pede, a partir das instruções acima:

1.1) (0,2 ponto [Dados]) Informe sua matrícula como uma lista ($matricula = [d1, d2, d3, d4, d5, d6, d7, d8, d9, d10]$), bem como a lista de parâmetros Dados;

1.2) (0,2 ponto [Dados]) Monte a integral a ser resolvida em r (ver (II)) e destaque a função v ;

1.3) (0,3 ponto [Dados]) Informe as tabelas em (III) em forma de lista e plote os diagramas de dispersão associados;

1.4) (0,3 ponto [Dados]) Informe o PVI em (IV) para os parâmetros associados à sua matrícula;

2) (1,0 ponto) Sobre o Item (II) e a Questão 1.2, faça o que se pede:

2.1) (0,25 ponto [Dados]) Plote o campo de direções tridimensionais no domínio D_{xz} , bem como a superfície

$v_y(r, \theta) = v(r) = b \left(1 - \frac{r}{r_0}\right)^{1/7}$ definida nesse domínio;

2.2) (0,25 ponto [Dados]) Plote as funções $v(r)$ e $rv(r)$ confrontando com os dados tabelados no intervalo completo;

2.3) (0,5 ponto [Dados]) Informe o resultado da integral em r com 9 casas de precisão e informe Q ;

3) (1,5 pontos) A partir dos dados em 1.3) e (III), faça o que se pede:

3.1) (0,5 ponto) Utilize o MMQ para encontrar os ajustes funcionais nos respectivos intervalos dados pelas tabelas acima;

(Sugestão: Utilize um ajuste quadrático para o primeiro subintervalo e um ajuste exponencial para o segundo subintervalo.)

3.2) (0,5 ponto) Plote essas representações confrontando com os Diagramas de Dispersão dados;

3.3) (0,5 ponto) Com as informações obtidas em 3.1, calcule as integrais nos respectivos subintervalos e informe Q dessa aproximação;

4) (1,0 ponto) Resolva o PVI (1.4) (ver (IV)) pelo Método de Heun e apresente uma tabela, de 10 em 10 valores das soluções, que confronte a solução numérica com a solução analítica, bem como o erro real associado. Faça um gráfico comparando a solução analítica *versus* a solução numérica e as linhas de fluxo associadas;

QUESTÕES - PARTE B

DADOS PARA A PARTE B

(I) Com as informações da matrícula:

- $aux = |d9 - d10| = |8 - 2| = 6$, e se $aux = 0$, considere $aux = 2$;
- $p = \left\lceil \frac{d1 + \dots + d4}{aux} \right\rceil = \left\lceil \frac{0+0+0+3}{6} \right\rceil = \lceil 0,5 \rceil = 1$;
- $q = \left\lfloor \frac{d5 + d6 + d7 + d8}{aux} \right\rfloor = \left\lfloor \frac{3+3+2+8}{6} \right\rfloor = \lfloor 2,67 \rfloor = 2$;
- $DADOS = [aux, p, q] = [6, 1, 2]$;

Montar a função f definida em $(-\infty, +\infty)$:

$$f(x) = \exp(-p|x|) \cos(q|x|) \overset{DADOS}{\Leftrightarrow} f(x) = \exp(-|x|) \cos(2|x|).$$

$$\Leftrightarrow f(x) = \exp(-|x|) \cos(2x)$$

(Gráfico via [WolframAlpha](#) e a [simplificação da paridade](#) de cos.)

Uma aplicação dessa função está descrita no artigo: Uma revisão teórica sobre funções de Autocorrelação aplicadas a altas e baixas Velocidades do vento (DEGRAZIA,2014)[¹].

(II) Montar a tabela:

- Tabela (A):

x_k	-2	-1	0	1	2
$f(x_k)$	$f(-2) = -0,08846$	$f(-1) = -0,15309$	$f(0) = 1$	$f(1) = -0,15309$	$f(2) = -0,08846$

(III) Com as informações de Dados da Parte A e o uso da diretriz sinal s :

"+" o elemento é um número par e "-" se o elemento é um número ímpar;

tem-se: $Dados = [a, b, c, d] = [3, 1, 3, 2] \Rightarrow Dados* = [-3, -1, , -3, +2];$

$$\text{Montar o PVI: } \begin{cases} ay' = d + \frac{cx}{by}, & x \in [1, 2] \\ y(1) = 2 \quad (h = 0, 1) \end{cases} \Leftrightarrow \begin{cases} (-3)y' = (+2) + \frac{(-3)x}{(-1)y} \\ y(1) = 2 \end{cases}$$

(IV) Softwares auxiliares:

- Para gráficos segue a instrução da Parte A;
- Pode-se recorrer ao WolframAlpha para a QUESTÃO 02, na estimativa dos seguintes tópicos:
 - [Cálculo de derivadas superiores](#);
 - [Extremos Globais da função derivada obtida acima](#);

- **Cálculo das integrais envolvidas** em $R_4(f)$ (para ilustrar);
- Para a determinação da solução analítica do PVI recomenda-se os procedimentos computacionais apresentados e discutidos em aula;

(V) Para a elaboração de gráficos pode-se usar as bibliotecas **MATPLOLIB**, **SYMPY** ou **SYMPY(2)**. Ainda, a biblioteca NumPy (estende-se para SciPy, Pandas, ...) pode/deve ser empregada para a elaboração de gráficos e, para qualquer outra finalidade, quando no enunciado do exercício for especificado o uso ou quando não for substituinte do objetivo fim do exercício;

(VI) Bibliografia.

[1]: DEGRAZIA *et al.* Uma revisão teórica sobre funções de Autocorrelação aplicadas a altas e baixas Velocidades do vento. Ciência e Natura, Santa Maria, vol. 36, Ed. Especial, pg. 101-107, março, 2014. Disponível em: < https://periodicos.ufsm.br/cienciaenatura/article/view/13222/pdf_1 >. Acesso em: 14/05/2023.

1) (2,0 ponto) A partir dos procedimentos em (II), faça o que se pede:

1.1) (1,0 ponto) Calcule o valor da integral sobre os dados da Tabela (A) e informe o erro da aproximação para algum dos Métodos de Newton-Cotes;

1.2) (1,0 ponto) Calcule o valor da integral sobre a lei funcional f , empregando a implementação do Método de Newton-Cotes feita em aula, garantindo uma precisão de 5 casas decimais;

2) (2,0 pontos) A partir dos procedimentos em (I), faça o que se pede:

2.1) (1,0 ponto) Considere $x \in [-2, 2]$ e os dados da Tabela (A) e monte uma aproximação polinomial de quarto grau para representar a função f e monte um gráfico comparativo;

2.2) (1,0 ponto) Considere $x \in [-2, 2]$ e a Tabela (A) com a inserção dos pares ordenados $(\pm 1, 5; f(\pm 1, 5))$ e $(\pm 0, 5; f(\pm 0, 5))$ para montar a aproximação polinomial de quarto grau para representar a função f e monte um gráfico comparativo;

3) (1,5 pontos) A partir dos procedimentos em (III), resolva numericamente o PVI (III) pelo Método Clássico de Runge-Kutta de 4ª Ordem (empregar o algoritmo desenvolvido em aula) e depois compare graficamente a solução numérica com a solução analítica (informe/printe essa solução), bem como as linhas de fluxo associadas.

SUGESTÕES.

(i) Sugere-se colocar a seguinte organização na confecção dos arquivos respostas:

AP(XX) - PARTE X

...

2) (2,0 pontos) A partir dos procedimentos em (I), faça o que se pede:

2.1) (1,0 ponto) ...

Inicia-se a resolução

⋮

2.1) (1,0 ponto) ...

⋮

(ii) Na questões relativas ao MMQ, utilizar a seguinte diretriz de resolução (conforme apresentado em aula):

- Apresentar tabela (caso discreto) ou função (caso contínuo) e respectiva representação gráfica;
 - Apresentar Sistema Normal e as seguintes informações associadas:
 - os produtos internos do caso discreto ou para o caso contínuo a resolução das integrais resolvidas via Método de Newton-Cotes conforme esquemas apresentados em aula;
 - dados decorrentes à Decomposição de Cholesky para obtenção da solução (usar algoritmos indicados e discutidos em aula);
 - A partir da solução obtida, informar a lei funcional do ajuste obtido;
 - Confrontação gráfico dos dados iniciais com o ajuste funcional obtido;
-

OBSERVAÇÃO: Colocar os documentos IPYNB e PDF da resolução no espaço TAREFA do MOODLE aberta, caso queira entregar o arquivo-resposta com esses recursos computacionais. Caso se resolva via Google COLAB, colocar link compartilhado sem restrições de acesso e também o arquivo PDF associado.

```
In [ ]: import numpy as np
import math
from math import trunc
import matplotlib.pyplot as plt
```

```
In [ ]: d1 = 0
d2 = 2
d3 = 0
d4 = 2
d5 = 2
d6 = 1
d7 = 0
d8 = 0
d9 = 7
d10 = 1

aux = abs(d9 - d10)
if aux == 0:
    aux = 2

p = math.ceil((d1 + d2 + d3 + d4)/aux)
q = math.floor((d5 + d6 + d7 + d8)/aux)
if q == 0:
    q = 1

print("Matrícula: %s"%(d1, d2, d3, d4, d5, d6, d7, d8, d9, d10))
print("Dados: %s"%(aux, p, q))
```

```
Matrícula: [0, 2, 0, 2, 2, 1, 0, 0, 7, 1]
Dados: [6, 1, 1]
```

```
In [ ]: f = lambda x: np.exp(-p*abs(x))*np.cos(-q*abs(x)) #usei o numpy para facilitar trabalhar com vetores
np.set_printoptions(precision=5)
xk = np.array([-2, -1, 0, 1, 2]) #fiz um array no np para calcular essa funcao mais rapidamente
```

```

fxk = f(xk)

print("xk: \t %.5f %.5f %.5f %.5f %.5f"%(xk[0], xk[1], xk[2], xk[3], xk[4]))
print("f(xk): \t %.5f %.5f %.5f %.5f %.5f"%(fxk[0], fxk[1], fxk[2], fxk[3], fxk[4]))

```

```

xk:      -2.00000 -1.00000 0.00000 1.00000 2.00000
f(xk):   -0.05632 0.19877 1.00000 0.19877 -0.05632

```

Dados da parte A, já com a diretriz do sinal:

$$\begin{aligned}
 a &= (+2), \\
 b &= (-1), \\
 c &= (+4), \\
 d &= (+8).
 \end{aligned}$$

$$\text{PVI obtido: } \begin{cases} ay' = d + \frac{cx}{by}, & x \in [1, 2] \\ y(1) = 2 \quad (h = 0, 1) \end{cases} \Leftrightarrow \begin{cases} (+2)y' = (+8) + \frac{(+4)x}{(+1)y} \\ y(1) = 2 \end{cases}$$

In []: *#código de newton cotes*

```

def newtoncotes(f, a, b, n, m):
    d = [2., 6., 8., 90., 288., 840., 17280., 28350.]
    c = [1., 1., 4., 1., 3., 7., 32., 12., 19., 75., 50., 41., 216., 27., 272., 751., 3577., 1323., 2989., 28350.,
    conderro = 0
    intgr = 0

    if n < 1 or n > 8:
        conderro += 1

    if m % n != 0:
        conderro += 2

    if conderro != 0:
        print("condERRO = %s"%conderro)
        return None

```

```

p = trunc(0.25*(n*(n + 2) + (n%2)))
h = (b - a)/m

for i in range(m+1):
    x = a + i*h
    y = f(x)
    j = p + trunc(0.5*n - abs((i%n) - 0.5*n))
    k = 1 + trunc((n - (i%n))/n) - trunc((m - (i%m))/m)
    intgr += y*c[j]*k

intgr *= n*h/d[n-1]

return intgr

```

Resposta 1.1

In []: *#para não ter que implementar novamente o código de newton cotes, é so notar que m é o número de subintervalos, #no caso da tabela m = 4.*

```

I = newtoncotes(f, -2, 2, 1, 4)
print(I)

```

1.3412128707006978

Obtemos:

$$I \approx 1.34121$$

comparado com o valor obtido com integração analítica (via Wolfram Alpha):

$$I = 1.17938$$

(arredondei a última casa).

Resposta 1.2

In []: `I = newtoncotes(f, -2, 2, 1, 1000)`
`print(I)`

1.1793818634911157

Obtemos uma precisão de 5 casas utilizando 1000 subintervalos:

$$I = 1.17938$$

```
In [ ]: #código de sistemas lineares
def LUdecomp(A):
    n = A.shape[0]
    m = A.shape[1]

    pivot = np.zeros(n)
    for i in range(n):
        pivot[i] = i+1

    det = 1
    for j in range(0,n-1):
        p = j
        Amax = abs(A[j,j])
        for k in range(j+1,n):
            if abs(A[k,j]) > Amax:
                Amax = abs(A[k,j])
                p = k

        if p != j:
            for k in range(0,m):
                t = A[j,k]
                A[j,k] = A[p,k]
                A[p,k] = t

            s = pivot[j]
            pivot[j] = pivot[p]
            pivot[p] = s
            det = -det

    det = det*A[j,j]
    if A[j,j] != 0:
        r = 1/A[j,j]
        for i in range(j+1,n):
            mult = A[i,j]*r
```

```

        A[i,j] = mult
        for k in range(j+1,m):
            A[i,k] = A[i,k] - mult*A[j,k]

    det = det*A[n-1,n-1]
    return (A, det, pivot)

def getL(LU):
    n = LU.shape[0]
    m = LU.shape[1]

    L = np.zeros((n,m))
    for i in range(n):
        for j in range(m):
            if i == j:
                L[i,j] = 1
            elif i > j:
                L[i,j] = LU[i,j]

    return L

def getU(LU):
    n = LU.shape[0]
    m = LU.shape[1]

    U = np.zeros((n,m))
    for i in range(n):
        for j in range(m):
            if i <= j:
                U[i,j] = LU[i,j]

    return U

def solveL(L, pivot, b):
    n = L.shape[0]

    y = np.zeros(n)
    k = int(pivot[0])
    y[0] = b[k-1]
    for i in range(1,n):
        soma = 0

```

```

        for j in range(0,i):
            soma = soma + L[i,j]*y[j]

        k = int(pivot[i])
        #tive que fazer essa adaptação porque o algoritmo do material não é genérico o suficiente
        #para permitir que seja qualquer matrix triangular inferior
        y[i] = (b[k-1] - soma)/L[i,i]

    return y

def solveU(U, y):
    n = U.shape[0]
    x = np.zeros(n)

    x[n-1] = y[n-1]/U[n-1,n-1]
    for i in reversed(range(0,n-1)):
        soma = 0
        for j in range(i+1,n):
            soma = soma + U[i,j]*x[j]

        x[i] = (y[i] - soma)/U[i,i]

    return x

def solveLU(A, b):
    LU, det, pivot = LUdecomp(A)
    L = getL(LU)
    U = getU(LU)
    y = solveL(L, pivot, b)
    x = solveU(U, y)
    return x

def choleskydecomp(A):
    n = A.shape[0]

    conderro = 0
    det = 1
    r = 1
    t = 1
    soma = 0
    for j in range(0,n):

```

```

soma = 0
for k in range(0,j):
    soma = soma + A[j,k]**2

t = A[j,j] - soma
if t > 0:
    A[j,j] = math.sqrt(t)
    r = 1/A[j,j]
    det = det*t
else:
    conderro = 1
    print("Erro: matriz não definida positiva")
    return (A, conderro, det)

for i in range(j+1,n):
    soma = 0
    for k in range(0,j):
        soma = soma + A[i,k]*A[j,k]

    A[i,j] = (A[i,j] - soma)*r
    #algoritmo no material retornava uma matriz que não era triangular inferior (ele não mexia nos elementos
    A[j,i] = 0

return (A, conderro, det)

```

```

def solvecholesky(A, b):
    G, conderro, det = choleskydecomp(A)
    #cholesky não faz pivoteamento, pivot vai ser só [1,2,3]
    pivot = np.arange(1, A.shape[0]+1)
    y = solveL(G, pivot, b)
    x = solveU(G.T, y)
    return x

```

```

def solvegauss_seidel(A, b, tol, itermax):
    n = A.shape[0]
    x = np.zeros(n)
    v = np.zeros(n)
    for i in range(0,n):
        r = 1/A[i,i]
        for j in range(0,n):
            if i != j:

```

```

        A[i,j] = A[i,j]*r

    #A[i,i] = 0
    b[i] = b[i]*r
    x[i] = b[i]

iter = 0
normarel = 1
while iter <= itermax:
    iter = iter + 1
    for i in range(0,n):
        soma = 0
        for j in range(0,n):
            if i != j:
                soma = soma + A[i,j]*x[j]

        v[i] = x[i]
        x[i] = b[i] - soma

    normanum = 0
    normaden = 0
    for i in range(0,n):
        t = abs(v[i] - x[i])
        normanum = normanum + t**2
        normaden = normaden + abs(v[i])**2

    normarel = math.sqrt(normanum/normaden)
    if normarel <= tol:
        conderro = 0
    else:
        conderro = 1

return x, normarel, conderro

```

```

In [ ]: #código de ajuste funcional
#não tem um algoritmo de ajuste funcional explícito na apostila, mas utilizei o que foi descrito no moodle
def discretenorm(u, v):
    if len(u) != len(v):
        raise ValueError("Vectors must have the same length")

```

```

    return sum([u[i]*v[i] for i in range(len(u))])

def discretepolynfit(x, y, n):
    M = np.zeros((n+1,n+1))
    b = np.zeros(n+1)
    for i in range(n+1):
        for j in range(n+1):
            M[i,j] = discretenorm(x**i, x**j)

    b[i] = discretenorm(y, x**i)

    #talvez tenha outros métodos mais eficientes de resolver esse sistema, mas como o LU e gauss compacto
    #ta funcionando bem aqui eu vou manter
    return solveLU(M, b)

def discreteexpfit(x, y):
    M = np.zeros((2,2))
    b = np.zeros(2)
    for i in range(2):
        for j in range(2):
            M[i,j] = discretenorm(x**i, x**j)

    b[i] = discretenorm(np.log(y), x**i)

    return solveLU(M, b)

```

Resposta 2.1

```

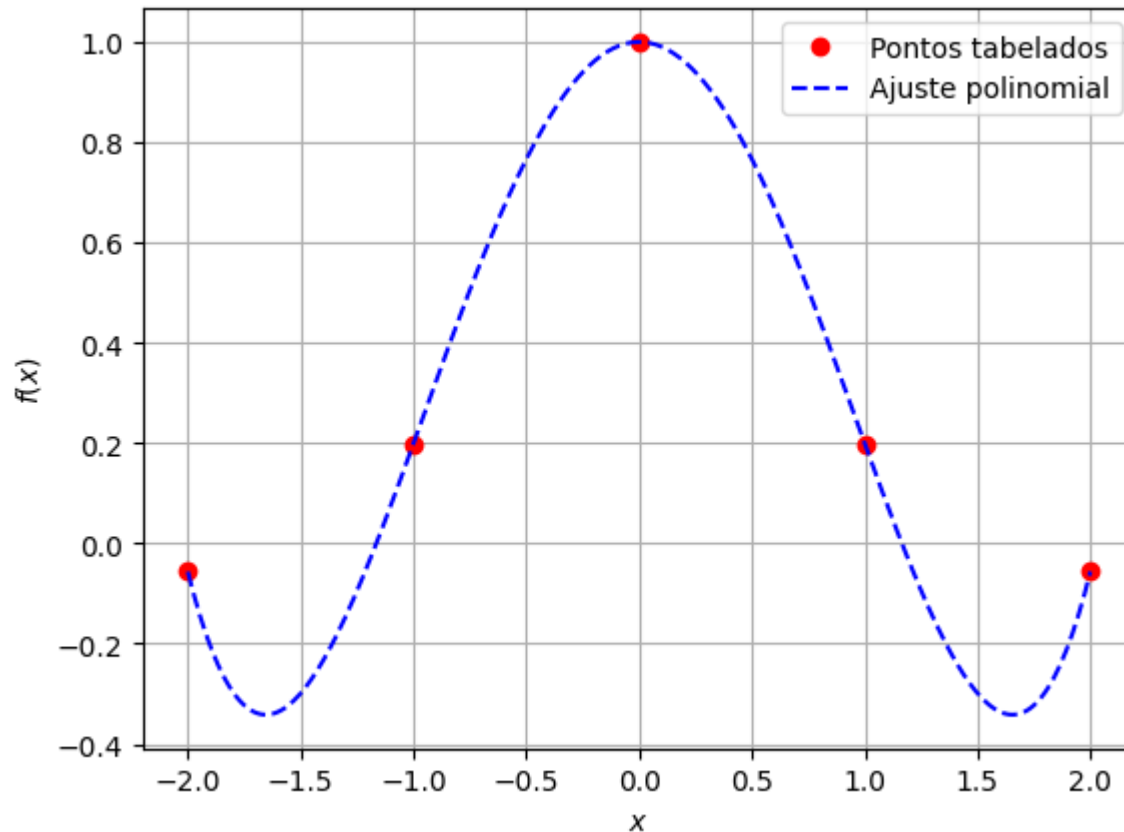
In [ ]: n = 4
coeff = discretepolynfit(xk, fxk, n)
print(coeff)
fpolyn = lambda x: sum([coeff[i]*x**i for i in range(0,n+1)])

xrange = np.linspace(-2, 2, 300)
plt.plot(xk, fxk, "ro")
plt.plot(xrange, fpolyn(xrange), "b--")
plt.xlabel("$x$")
plt.ylabel("$f(x)$")

```

```
plt.grid()  
plt.legend(["Pontos tabelados", "Ajuste polinomial"])  
plt.show()
```

```
[ 1.00000e+00 -1.31068e-17 -9.80285e-01  3.85494e-18  1.79051e-01]
```



Ajustamos a função por um polinômio de modo que:

$$f(x) \approx 1.000 - 1.311 \times 10^{-17}x - 9.803 \times 10^{-1}x^2 + 3.855 \times 10^{-18}x^3 + 1.791 \times 10^{-1}x^4$$

Resposta 2.2

```
In [ ]: xk2 = np.array([-2, -1.5, -1, 0, 1, 1.5, 2])
```

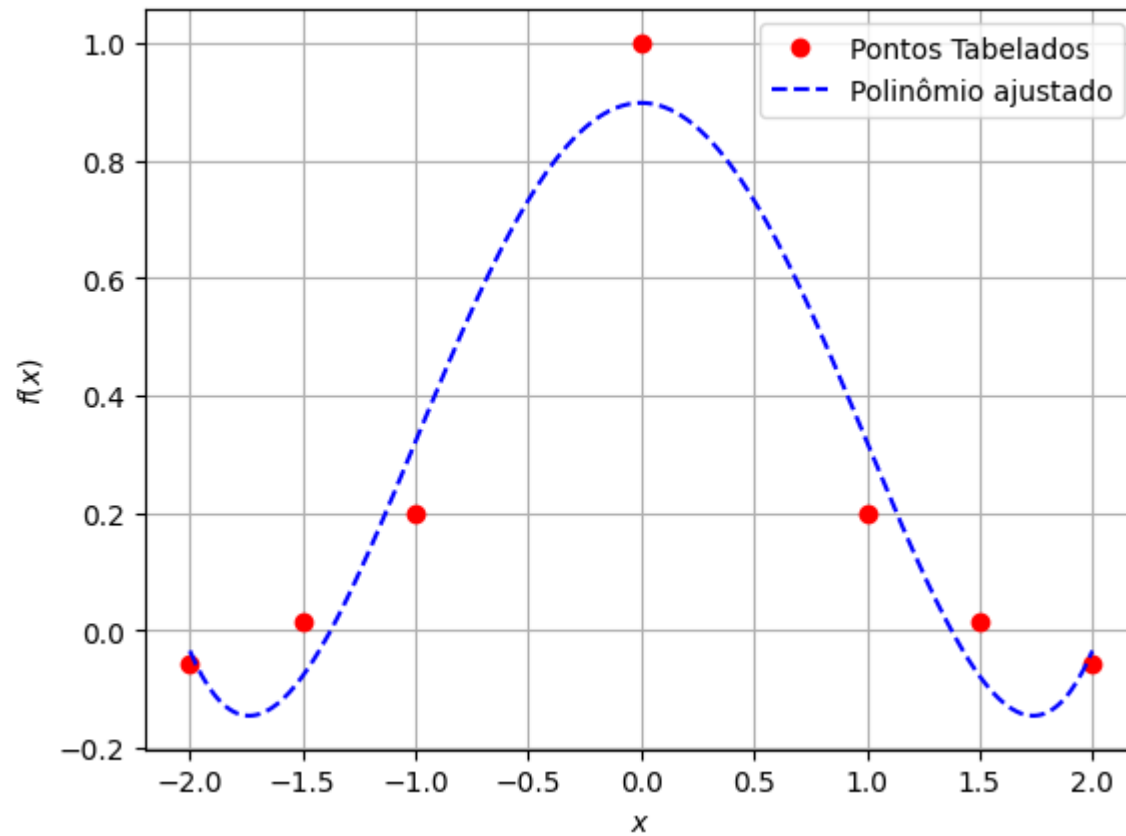
```

fxk2 = f(xk2)

coeff2 = discretepolyfit(xk2, fxk2, n)
print(coeff2)
fpolyn2 = lambda x: sum([coeff2[i]*x**i for i in range(0,n+1)])
plt.plot(xk2, fxk2, "ro")
plt.plot(xrange, fpolyn2(xrange), "b--")
plt.xlabel("$x$")
plt.ylabel("$f(x)$")
plt.grid()
plt.legend(["Pontos Tabelados", "Polinômio ajustado"])
plt.show()

```

[0.89795 0. -0.69125 -0. 0.11454]



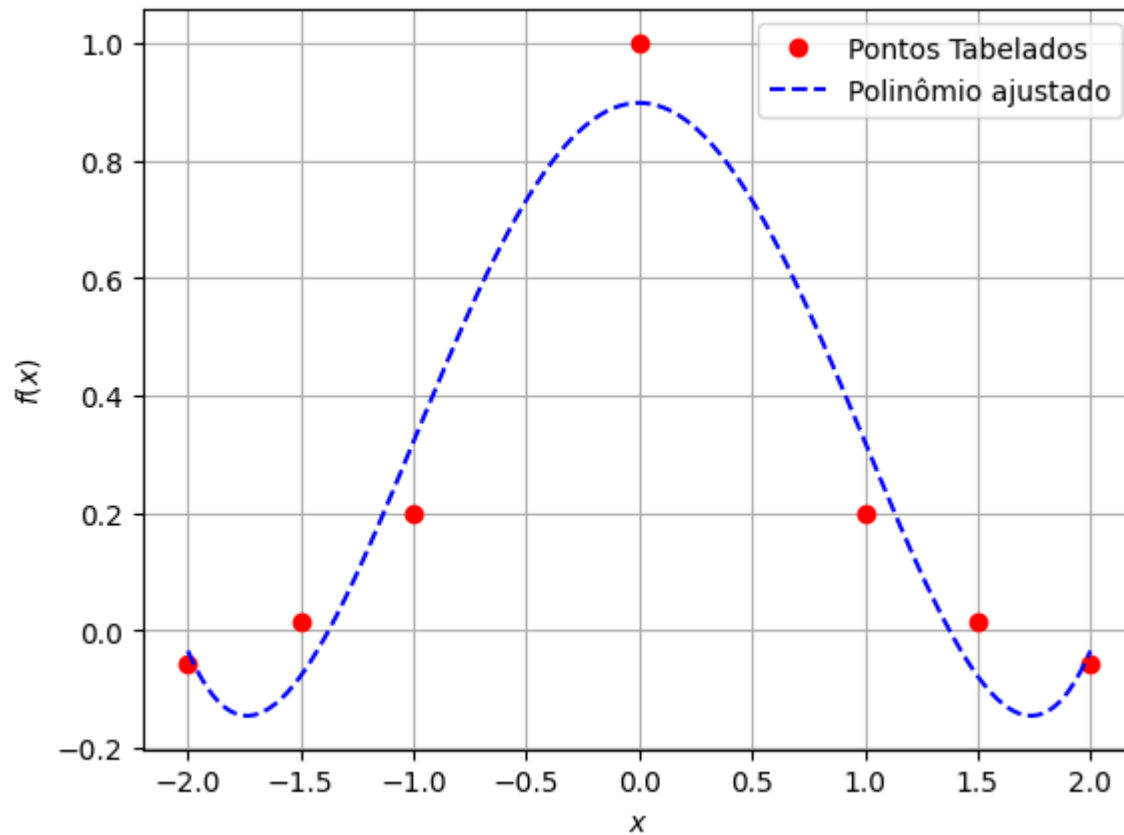
Aproximamos a função de modo que:

$$f(x) \approx 0.898 - 0.691x^2 + 0.115x^4$$

Enquanto os pontos tabelados originais eram bem descritos por uma função de quarto grau, os segundos descrevem melhor a função (por ter mais pontos), porém já vemos que também não são bem descritos por uma função de quarto grau (talvez precisariam de um grau superior).

```
In [ ]: n3 = 6

coeff3 = discretepolynfit(xk2, fxk2, n3)
fpolyn3 = lambda x: sum([coeff3[i]*x**i for i in range(0,n3+1)])
plt.plot(xk2, fxk2, "ro")
plt.plot(xrange, fpolyn2(xrange), "b--")
plt.xlabel("$x$")
plt.ylabel("$f(x)$")
plt.grid()
plt.legend(["Pontos Tabelados", "Polinômio ajustado"])
plt.show()
```



Um polinômio de ordem 6 também não ajustou muito bem.

```
In [ ]: #código feito para solução de edos
#não tem nenhum algoritmo explícito na apostila, mas utilizei o texto nela como base
def euler(f, x0, y0, h, n):
    #floor não funciona muito bem por causa de divisões estranhas de floats, ex: 0.3/0.1 = 2.9999999999999996
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        x[i] = x[i-1] + h
        y[i] = y[i-1] + h*f(x[i-1],y[i-1])
    i
```

```

    return (x,y)

def heun(f, x0, y0, h, n):
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        K1 = f(x[i-1], y[i-1])
        K2 = f(x[i-1] + h/3.0, y[i-1] + h*K1/3.0)
        K3 = f(x[i-1] + 2.0*h/3.0, y[i-1] + 2.0*h*K2/3.0)
        y[i] = y[i-1] + h*(K1 + 3.0*K3)/4.0
        x[i] = x[i-1] + h

    return (x,y)

#é facilmente generalizado para sistemas de equações diferenciais
def rk4(f, x0, y0, h, n):
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        K1 = f(x[i-1], y[i-1])
        K2 = f(x[i-1] + h/2.0, y[i-1] + h*K1/2.0)
        K3 = f(x[i-1] + h/2.0, y[i-1] + h*K2/2.0)
        K4 = f(x[i-1] + h, y[i-1] + h*K3)
        y[i] = y[i-1] + h*(K1 + 2.0*K2 + 2.0*K3 + K4)/6.0
        x[i] = x[i-1] + h

    return (x,y)

```

Resposta 3

$$\text{PVI obtido: } \begin{cases} ay' = d + \frac{cx}{by}, & x \in [1, 2] \\ y(1) = 2 \quad (h = 0, 1) \end{cases} \Leftrightarrow \begin{cases} (+2)y' = (+8) + \frac{(+4)x}{(+1)y} \\ y(1) = 2 \end{cases}$$

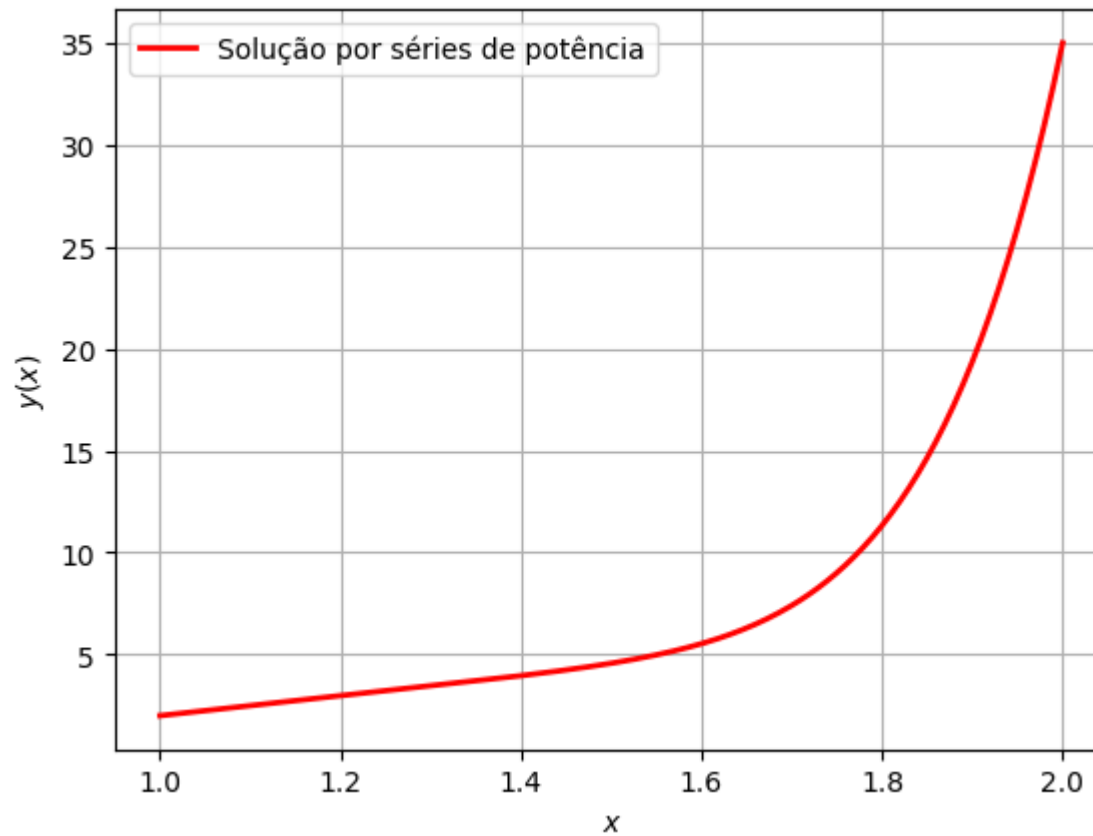
Tem solução em série de potências truncada com sympy.

```
In [ ]: from sympy import dsolve, Function, pprint, sin, cos, N
from sympy.abc import x
import sympy.plotting as plt

y = Function("y")
eq = -2.0*y(x).diff(x, 1) + 8.0 + 4.0*x/y(x)
ysol = dsolve(eq, hint="1st_power_series", ics={y(1): 2}, n=8).rhs
print(ysol)

ysoln = lambda x: -3.0 - 0.75*(x - 1)**2 + 1.375*(x - 1)**3 - 2.890625*(x - 1)**4 + 6.5859375*(x - 1)**5 - 15.82617
xrange = np.linspace(1, 2, 300)
plt.plot(xrange, ysoln(xrange), "r-", lw=2)
plt.xlabel("$x$")
plt.ylabel("$y(x)$")
plt.grid()
plt.legend(["Solução por séries de potência"])
plt.show()

-3.0 - 0.75*(x - 1)**2 + 1.375*(x - 1)**3 - 2.890625*(x - 1)**4 + 6.5859375*(x - 1)**5 - 15.826171875*(x - 1)**6 + 3
9.5006975446429*(x - 1)**7 + 5.0*x + 0(x**8)
```



```
In [ ]: df = lambda x,y: 4.0 + 2.0*x/y
x0 = 1.0
fx0 = 2.0
h = 0.1
n = 10
xsol,ysol = rk4(df, x0, fx0, h, n)

xmesh = np.linspace(0.0, 2.2, 100)
ymesh = np.linspace(2, 8, 100)

Xl, Yl = np.meshgrid(xmesh, ymesh)
P = Xl/Xl
Q = 4.0 +2.0*Xl/(Yl)
```

```

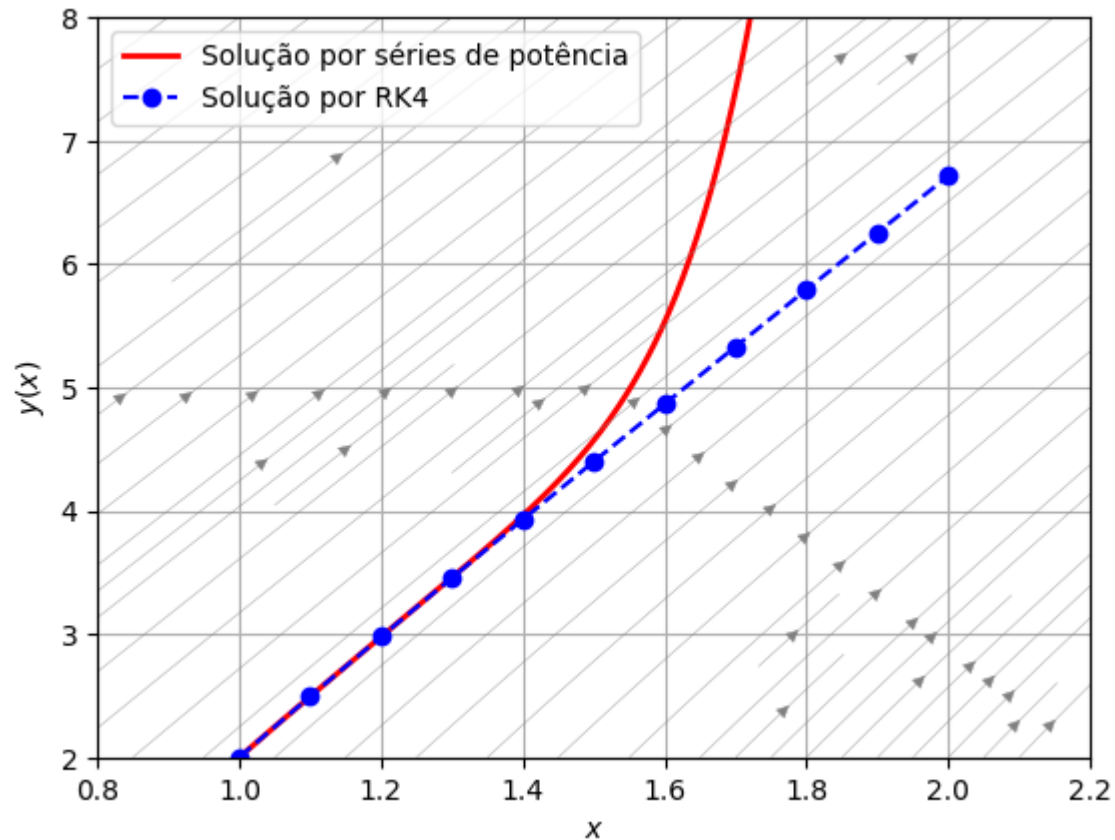
plt.plot(xrange, ysoln(xrange), "r-", lw=2)
plt.plot(xsol, ysol, "bo--")

plt.streamplot(Xl, Yl, P, Q, density=1.5, color="gray", linewidth=0.25)

plt.xlabel("$x$")
plt.ylabel("$y(x)$")
plt.xlim(0.8, 2.2)
plt.ylim(2, 8)
plt.grid()
plt.legend(["Solução por séries de potência", "Solução por RK4"])
plt.show()

```

/tmp/ipykernel_5546/2636859275.py:12: RuntimeWarning: invalid value encountered in divide
P = Xl/Xl



Temos uma aproximação boa em série de potência somente na região perto de 1, que foi onde a solução é expandida em série de potência, enquanto a solução numérica se aproxima bem da solução real.